



Figure 1: First program code

Use of Python in scripting Sikuli

To enhance the scripting capabilities, we can access the entire Python language. As an example, let's suppose we run our Sikuli script unattended, and it has failed. Then by using the *captureto()* method we can save the images that can be used to debug script failure.

Accessing Java from Sikuli

To provide some kind of on-screen display, we can use Java classes and, in this way, give a GUI representation to our script. Sikuli starts by importing the Swing classes, which are some of Java's GUI libraries, and then uses Swing to show a borderless window with the designated text over everything on the screen for the specified number of seconds (the default is 1 second).

Start your first script with Sikuli

To begin with, let's build a very basic and simple 'Hello world' example of a testing script for an application WordPad to understand what a simple script looks like. Figure 1 shows how our complete script will look.

First of all, start Sikuli, select the editor and write the following line of code:

```
App = App.open(r'c:\Program Files\Windows NT\Accessories\wordpad.exe')
```

This will start the WordPad application using Sikuli.

Note: The character *r* in this code is to use space in the application path, if any.

Type 'wait' in the editor, and then click on the *Take Screenshot* button before selecting the area of screen that you would like Sikuli to wait for the text to appear.

```
Wait (screenshot1.png)
```

Now add input text to the WordPad application using *type()*, as follows:

```
type ("Hello, this is my first Sikuli Code!")
```

You can check whether the text that you entered appears as expected or not, using the *wait()* command:

```
Wait (Hello, this is my first Sikuli Code!)
```

Now save and run the script. It will open the WordPad, and write input text to it. In this way, you can start coding and further customise it according to your requirement.

A trick you can try out

If we get two similar buttons like the *Save* button on the same screen and the script is unable to distinguish between the two prior to selecting which one is to be clicked, use a larger portion of the screen in scripting. This will help Sikuli to get a better understanding of the buttons.

Advantages and disadvantages of Sikuli

Pros:

- Sikuli is an open source tool, so it is better than tools like UFT.
- It works very well with Flash objects.
- It is very handy in automation when working with Web elements that have dynamic Xpaths and IDs.
- It has multiple scripting and programming language support like Scala, JRuby and Jpython.
- It is very good with boundary value analysis testing.
- With proper and smart use of scripting, it is easy to identify application crashes and bugs.
- The Sikuli set-up and use is very simple and easy.
- Automation testing of mobile applications can also be done with the help of emulators.
- Its integration with Selenium makes it worth using. It can solve the browser dialogue box handling problems of Selenium.
- It can read texts on images with the help of its basic text recognition OCR.
- It supports almost every platform including Windows XP+, most Linux flavours and Mac OS 10.5+.
- Sikuli is very useful in functional testing where input and output are predefined; so it can be used efficiently for testing the overall behaviour of applications.

Cons:

- The script cannot be run on the back-end as it needs a visible application GUI during the time it is being executed.
- It is platform and resolution dependent.
- Running multiple scripts automatically, one by one, is very tricky in Sikuli.
- A slight change in the text label or image of the GUI of the application can result in the failure of the script.
- Maintenance of scripting is very hard if the GUI of the apps changes frequently. 

By: Aakash Beniwal

The author works with Infosys Limited, Pune, as a testing engineer and has over two years' experience in this domain. He can be reached at aakashnavodaya@gmail.com.